

Elephant Movement Trajectory Forecast

Benchmarking CNN, RCNN, and LSTM Architectures
for Spatiotemporal GPS Trajectory Prediction

Pennsylvania State University

Legacy Trajectory Model

Dataset: ~780,000 GPS-collar records

Region: African elephants, Okavango Delta / Etosha

Test rows per model: 117,015

Features: 18 input → 2 output (lon, lat)

Window: seq_len=20, pred_len=1

Quick Results Summary

Model	Params	Haversine (m)	Lon MAE (m)	Lat MAE (m)
CNN	57,154	120.91	95.47	62.12
RCNN	70,082	487.68	419.27	212.24
LSTM	216,387	396.47	350.15	150.34

Dataset Structure

Source

GPS collar tracking data from African elephants in the Okavango Delta / Etosha region (Botswana / Namibia). The raw dataset contains ~780,000 rows.

Preprocessing Pipeline

- 1. Sort by (elephant_id, timestamp) — ensures temporal order per animal
- 2. Encode time_of_day as ordinal: night=0, morning=1, afternoon=2, evening=3
- 3. Fill NaN with forward-fill then backward-fill per elephant; remaining NaN → 0
- 4. Optional subsampling (`--sample_frac < 1.0`) per elephant for prototyping

Feature Columns (18 total → 2 targets)

Group	Features
Position	longitude, latitude
Movement	speed_kmh, step_dist_m, turning_angle, bearing, dir_persistence, acceleration
Temporal	hour, season_code, time_of_day_code
Environmental	NDVI, EVI, LST_celsius, elevation_m, slope_deg, water_occ_1km, NDWI
Target	longitude, latitude (next step)

Window Construction & Splits

- Grouped by elephant_id — windows never cross between animals
- Sliding window: seq_len=20 steps → X, pred_len=1 step → y
- Feature scaling: StandardScaler fit on all 18 features combined
- Target scaling: Separate StandardScaler fit on (lon, lat) only
- Split: 70% train / 15% validation / 15% test (random_state=42, window level)

Data Modes

Mode	Data File	Approx. Rows
test	sample_data.csv	~80,000
real	actual_data.csv	~780,000

CNN Forecaster Architecture

Design Philosophy

Pure convolutional approach — treats the sequence as a 1-D signal.
Architecture archetype: WaveNet / TC-ResNet
Trainable Parameters: 57,154

Layer-by-Layer Flow

Input	(B, T=20, F=18)
Permute	(B, F=18, T=20)
Conv1d + ReLU	18 → 64 channels, kernel=3, padding=1
Conv1d + ReLU	64 → 128 channels, kernel=3, padding=1
Conv1d + ReLU	128 → 64 channels, kernel=3, padding=1
AdaptiveAvgPool1d	(B, 64, 1) — collapse time dimension
Flatten	(B, 64)
Linear + ReLU	64 → 64
Linear (Output)	64 → pred_len × out_dim = 1 × 2

Key Design Choices

- Three Conv1d layers with growing then shrinking channel dimensions (18→64→128→64)
- Same padding (padding=1) preserves the time dimension through convolutions
- AdaptiveAvgPool1d(1) collapses all 20 time steps into a single vector per channel
- This works best when the relevant signal is distributed across the whole window
- No recurrent layers — purely spatial feature extraction over time

Forward Pass Signature

```
def forward(self, x): # x: (B, seq_len, n_features)
    x = x.permute(0, 2, 1) # → (B, n_features, seq_len)
    x = self.conv(x)       # → (B, 64, seq_len)
    x = self.pool(x)       # → (B, 64, 1)
    return self.head(x).view(-1, self.pred_len, self.out_dim)
```

RCNN Forecaster Architecture

Design Philosophy

Hybrid CNN + RNN — convolutional feature extractor feeds into a GRU sequence encoder.
Architecture archetype: ConvLSTM / LRCN
Trainable Parameters: 70,082

Layer-by-Layer Flow

Input	(B, T=20, F=18)
Permute	(B, F=18, T=20)
Conv1d + ReLU	18 → 64 channels, kernel=3, padding=1
Conv1d + ReLU	64 → 64 channels, kernel=3, padding=1
Permute	(B, T=20, 64) — restore time as dim 1
GRU	input=64, hidden=64, layers=2, dropout=0.2
Hidden State	h[-1]: last layer's final hidden → (B, 64)
Linear + ReLU	64 → 64
Linear (Output)	64 → pred_len × out_dim = 1 × 2

Key Design Choices

- Two Conv1d layers with same channel count (18→64→64) act as feature projector
- GRU receives 64-dim feature vector per time step for sequence encoding
- Only h[-1] (final hidden state) is used — discards per-step GRU outputs
- This forces the GRU to compress the entire sequence into one state vector
- Dropout=0.2 between GRU layers helps prevent overfitting

Forward Pass Signature

```
def forward(self, x):
    x = x.permute(0, 2, 1) # (B, n_feat, T)
    x = self.cnn(x).permute(0, 2, 1) # (B, T, 64)
    _, h = self.gru(x) # h: (layers, B, hidden)
    return self.head(h[-1]).view(-1, self.pred_len, self.out_dim)
```

LSTM Forecaster Architecture (with Attention)

Design Philosophy

Stacked LSTM with learnable attention-weighted output.
Architecture archetype: DA-RNN / Bahdanau-style attention
Trainable Parameters: 216,387

Layer-by-Layer Flow

Input	(B, T=20, F=18)
LSTM	input=18, hidden=128, layers=2, dropout=0.2
LSTM Output	out: (B, T=20, 128)
Attention Score	Linear(128→1) → (B, T, 1)
Softmax	over time dim → convex weights (B, T, 1)
Context Vector	$\Sigma(\text{weight} \times \text{out}) \rightarrow (B, 128)$
Linear + ReLU	128 → 64
Linear (Output)	64 → pred_len × out_dim = 1 × 2

Key Design Choices

- Stacked LSTM (2 layers) captures multi-scale temporal dependencies
- Attention learns a scalar weight per time step via Linear(128→1)
- Softmax over time produces a convex combination — focuses on relevant steps
- Unlike RCNN (uses only h[-1]), attention uses all LSTM outputs weighted by importance
- Useful for animal movement where recent context varies by behaviour state
- Dropout=0.2 between LSTM layers for regularisation

Forward Pass Signature

```
def forward(self, x):
    out, _ = self.lstm(x) # (B, T, hidden)
    weights = torch.softmax(self.attn(out), dim=1) # (B, T, 1)
    ctx = (weights * out).sum(dim=1) # (B, hidden)
    return self.head(ctx).view(-1, self.pred_len, self.out_dim)
```

Training Protocol

Hyperparameters (shared across all models)

Hyperparameter	Value
Loss function	HuberLoss (smooth L1) — robust to GPS outliers
Optimizer	Adam (lr=1e-3)
LR scheduler	ReduceLROnPlateau (patience=5, factor=0.5, monitor=val_loss)
Gradient clipping	max_norm = 1.0
Epochs	40 (configurable)
Batch size	64 (configurable)
Model selection	Checkpoint with lowest validation loss

Per-Epoch Training Loop

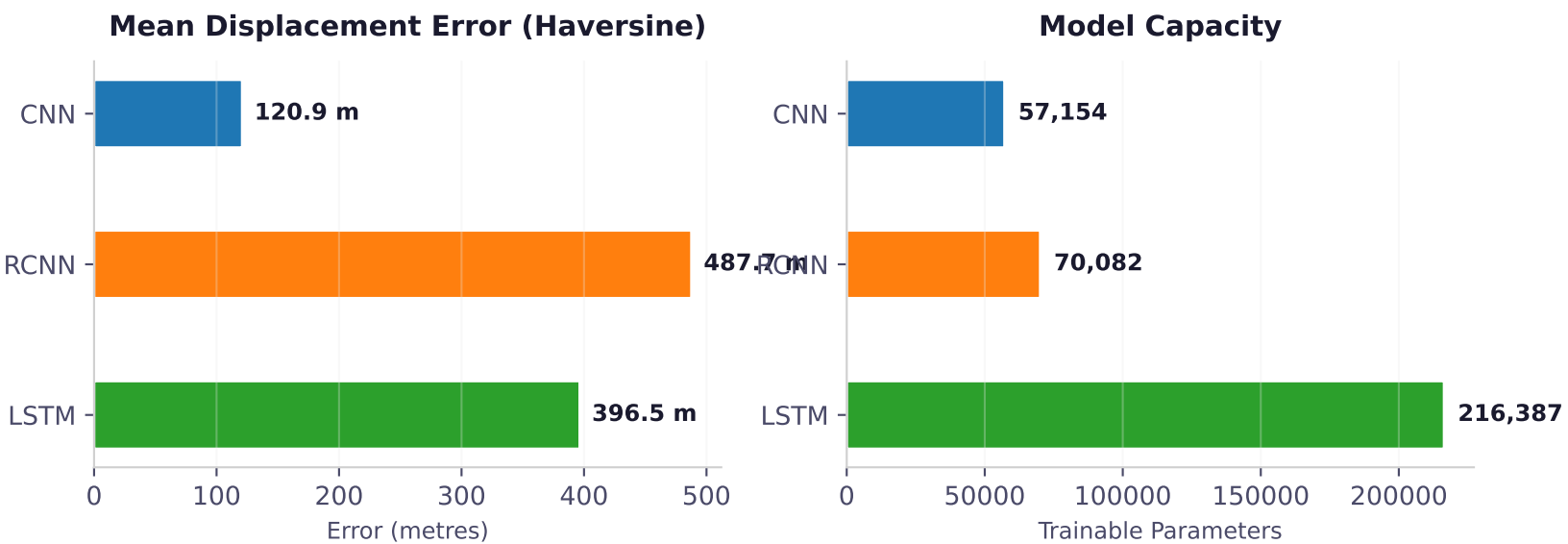
- Forward pass: `pred = model(X_batch)`
- Compute loss: `HuberLoss(pred, y_batch)`
- Backward pass + gradient clipping (`max_norm=1.0`)
- Optimizer step (Adam update)
- Validate: compute `val_loss` and Haversine displacement (metres)
- Inverse-transform predictions via target `StandardScaler`
- LR scheduler step on `val_loss`
- Save best checkpoint if `val_loss` improved

Evaluation Metrics

- HuberLoss**
Smooth L1 loss: quadratic for small residuals, linear for large ones. Less sensitive to GPS outliers than MSE.
- Haversine Distance**
Great-circle distance between predicted and true (lon, lat) in km, converted to metres. Accounts for Earth's curvature (R=6,371 km).
- Mean Displacement Error**
Mean Haversine distance in metres across all test predictions. Primary ranking metric.

Haversine Formula

Results & Analysis



Detailed Test Results

Model	Parameters	Haversine (m)	Lon MAE (m)	Lat MAE (m)	Test Rows
CNN	57,154	120.91	95.47	62.12	117,015
RCNN	70,082	487.68	419.27	212.24	117,015
LSTM	216,387	396.47	350.15	150.34	117,015

Analysis

- ### 1. CNN is the clear winner

Achieves the lowest mean displacement error of 120.91m with the fewest parameters (57K). This suggests elephant movement patterns are primarily spatial — well captured by convolution over the time axis — rather than requiring complex temporal state tracking.
- ### 2. RCNN underperforms

Worst performance at 487.68m error despite 70K parameters. The GRU's reliance on only the final hidden state ($h[-1]$) discards information from earlier timesteps, and the shallow CNN feature extractor (2 layers) may not capture enough spatial complexity.
- ### 3. LSTM with attention ranks second

396.47m error with 216K parameters — the largest model. The attention mechanism helps weight important timesteps, but the excessive capacity relative to CNN suggests potential overfitting. The model is $\sim 3.8\times$ larger than CNN for $\sim 3.3\times$ worse accuracy.
- ### 4. Parameter efficiency matters

CNN achieves the best accuracy with only 26% of LSTM's parameters, making it the most efficient architecture. For production deployment, CNN offers the best accuracy-efficiency tradeoff.
- ### 5. Error patterns

The ranking (CNN > LSTM > RCNN) holds across all metrics: Haversine distance, longitude MAE, and latitude

Comparison & Conclusions

Architecture Comparison

Aspect	CNN	RCNN	LSTM
Approach	Pure Conv1d	Conv1d + GRU	LSTM + Attention
Parameters	57,154 (smallest)	70,082	216,387 (largest)
Haversine Error	120.91m ✓ best	487.68m ✗ worst	396.47m
Longitude MAE	95.47m ✓ best	419.27m	350.15m
Latitude MAE	62.12m ✓ best	212.24m	150.34m
Temporal Modeling	Global avg pool	GRU hidden state	Attention weighting
Strengths	Fast, efficient, good spatial features	Combines local + temporal patterns	Learns which steps matter most
Weaknesses	Uniform pooling may miss temporal dynamics	Shallow CNN, GRU discards early states	Overparameterised, may overfit
Best For	Distributed spatial signal across window	Sequential pattern with local features	Variable-length context importance

Key Findings

- CNN is the best architecture for elephant trajectory forecasting in this dataset. It achieves the lowest mean displacement error (120.91m) with the fewest parameters (57K), suggesting elephant movement patterns are primarily spatial rather than requiring complex temporal state tracking.
- Attention (LSTM) provides moderate improvement over RCNN but is hampered by excessive parameters (216K). The attention mechanism's ability to weight timesteps differently is beneficial, but the model capacity is mismatched to the data volume — it is 3.8× larger than CNN for 3.3× worse accuracy.
- RCNN underperforms despite its hybrid design. The shallow CNN (2 layers) and the GRU's reliance on the final hidden state may discard spatial and temporal information that the CNN and LSTM capture more effectively. Using only h[-1] discards per-step outputs, forcing full sequence compression.
- For production deployment, the CNN offers the best accuracy-efficiency tradeoff. It trains fastest, uses the least memory, has the fewest parameters, and produces the most accurate predictions.